1757: A Real-Time Multiplayer Social Deduction Game

SWE 4404 Software Project Lab II · Final Report · Group 3 · 28 October 2026

Front Matter

Project	1757: a real-time multiplayer social deduction game set at the Battle of Plassey
Course	SWE 4404, Software Project Lab II (SPL-II), A.Y. 2024-25
Submitted to	Faisal Hussain, Assistant Professor, Department of CSE, IUT; Farzana Tabassum, Lecturer, Department of CSE, IUT
Department	Computer Science and Engineering, Islamic University of Technology (IUT)
Programme	BSc in Software Engineering
Group	Group 3
Supervisor(s)	**Samia Nawsheen, Junior Lecturer, Department of CSE, IUT**
Repository	**https://github.com/SaifSarker11/1757-RPG**

Member	Student ID	Main area
Wasi Omar Syed	230042143	Game engine (finite state machine, voting, missions, roles, vision, endgame), rooms and Royal Decree mode, rate limiting, voice chat, background music
Md. Saif-Al Sarker	230042106	Monorepo scaffold and CI/CD, reconnection handling, web room page integration, users module, admin dashboard

Member	Student ID	Main area
M Safwan Hasan Khan	230042117	Authentication and guest play, real-time chat, first room flow, UI themes and profiles, sound effects, Docker

Table of Contents

1. Executive Summary
2. Problem Domain and Background
3. Literature and Existing Solutions
4. Requirements (SRS Session)
5. System Design (Design Session)
6. Implementation (Implementation Session)
7. Tools, Version Control, Testing, and CI/CD
8. Professional, Ethical, and Legal Considerations
9. Project Management and Feasibility
10. Teamwork and Individual Contributions
11. Results, Limitations, and Future Work
12. Glossary and References

1. Executive Summary

1757 is an online game that 6 to 15 friends can play together in a web browser, on their phones or on a computer. It is set in Bengal in 1757, around the Battle of Plassey, where the betrayal of the Nawab by his own court handed power to the East India Company.

The problem. Hidden-role party games such as Mafia, Werewolf and The Resistance are popular, but they usually need everyone in the same room, someone to act as moderator, and a physical card deck. Playing online means trusting a host to keep secrets, and few such games teach any local history.

The solution. In 1757, every player secretly receives the role of a real historical figure. Most belong to one of two sides:

- **The Nawab's Loyalists (Green)** want to protect Bengal, but they do not know who the traitors are.

- **The East India Company's conspirators (Red)** hide inside the court and secretly sabotage missions.

One character, the banker **Jagat Seth**, is an exception: he belongs to neither side and can win alone with a correct final guess. Over up to five rounds, players argue in chat or voice, choose a team, vote on it, and the chosen team secretly helps or sabotages the mission. The first side to win three rounds moves to a final guessing phase that can still change the result.

The computer does the moderator's job. It deals the roles, tells each player only what their character is allowed to know, runs the timers, counts the votes and keeps score, so nobody can cheat by peeking.

The outcome. We delivered a working, deployed system:

- a web game, an admin dashboard and an early mobile app (sign-in and chat);
- an admin dashboard showing live rooms, balance between the sides, and match logs;
- guest play and accounts with experience points (XP), levels and match history;
- 15 historical roles plus 2 simple filler roles;
- text and voice chat;
- protection against dropped connections, where a player has 30 seconds to rejoin and the game continues if they do not.

The game was demonstrated live with six players at the final presentation on 28 October 2026. 13 of the 19 functional requirements are complete and 6 are partly done. The main limitations are some working API features (kick, Quick Join) that have no buttons in the web app yet, a mobile app with sign-in and chat only, and a single game server that cannot yet be spread across several machines (Section 11).

2. Problem Domain and Background

1757 belongs to the domain of real-time multiplayer social deduction games: games where hidden information, persuasion and voting decide the winner, and the software must act as a fair, secret-keeping referee for many players at once.

Users.

User	What they need
Players (6–15 per room, registered or guests)	Join a room quickly, see only their own secret information, chat or talk, and get a fair result
Room hosts	Create public or private rooms, set the player cap, timers and allowed roles, and remove disruptive players
Administrators	See active rooms, user counts and win-rate balance between the sides, and moderate users
Students and history enthusiasts	An engaging way to meet the people and events of the Battle of Plassey

Why it matters.

- A game like this is a demanding software problem. Several people act at once over the network, and each must see a different, consistent slice of the same game. The server must never leak a secret, and the game must survive players disconnecting.
- It also turns a key moment of Bengal's history, the betrayal at Plassey, into an experience players remember. Every named role, such as Siraj ud-Daulah, Mir Jafar, Mir Madan and Jagat Seth, is a real historical figure.

CO1: Fundamental concepts behind the problem

Concept	Where it appears in 1757
Finite state machine	The match moves through 8 named states (Role Reveal, Discussion, Selection, Ballot, Resolution, Round Result, Endgame, Post-game) with guarded transitions
Pure functions and immutability	The engine is one function, `reduce(state, action, now, rng) → { state, events }`, which never modifies its input, so every match can be replayed and tested
Seeded pseudo-random number generation	Role dealing and card shuffling use a seeded RNG, so a test can reproduce any game exactly

Concept	Where it appears in 1757
Data structures	Maps from player to role and vision, ordered seat arrays for leader rotation, weighted vote tallies, a fail-track counter, and per-player view projections
Event-driven and client-server design	Clients send actions; the server is the single source of truth and pushes updates over WebSockets
Relational database design	PostgreSQL tables for users, rooms, matches, rounds and ballot votes, with foreign keys, cascading deletes and unique constraints
Caching and persistence	Live game snapshots in Redis with a time-to-live, and permanent history in PostgreSQL
Input validation and authorization	Every incoming message is checked with a schema, and the acting player is taken from the login session, never from the message

CO1: Specialist knowledge

- **Real-time networking.** Socket.IO rooms, private per-user channels, acknowledgements, reconnection and a 30-second grace window. A turn-based web form cannot deliver a live, timed game.
- **Information hiding in multiplayer games.** A `viewFor(state, player)` projection builds a different view for each role, with decoys (Tara Bai, Ghasheti Begum) and invisible roles (Rai Durlabh). Sending the full state to the browser would let anyone cheat by opening developer tools.
- **Game balance.** Team sizes, team mixes and sabotage thresholds by player count are adapted from established hidden-role game design and kept in one config file for tuning.
- **Peer-to-peer media.** WebRTC voice chat, where the server only relays connection setup messages, keeps voice latency low and server costs near zero.
- **History of the Battle of Plassey.** Each role's ability reflects the person's historical part. For example, Mir Jafar's betrayal is why he can steal a Green win by unmasking Mir Madan.

3. Literature and Existing Solutions

1757 builds on the team-vote-and-secret-mission core of The Resistance. It adds history-based roles with partial and deceptive vision, an endgame twist, and a fully automated online referee. We found no existing game that combines these.

Game	Year, origin	Players	Core mechanic	Gap 1757 addresses
Mafia / Werewolf	1986, Dmitry Davidoff, Moscow State University; Werewolf theme 1997	Group	Informed minority vs uninformed majority; night kills, day votes	Needs a non-playing human moderator; players are eliminated and sit out
The Resistance / Avalon	2010 / 2012, Don Eskridge	5–10	Leader picks a team, everyone votes, team plays secret Success/Fail cards; 5 rejections give Spies the win; first to 3 missions	Physical cards; roles are generic; vision is all-or-nothing
Secret Hitler	2016, Temkin, Boxleiter, Maranges	5–10	President nominates Chancellor, vote, policy cards, election tracker	Physical; a modern political theme rather than local history
Town of Salem	2014, BlankMediaGames	Online	Many secret roles, night abilities, day lynch votes, neutral roles	Online and automated, but elimination-based and unrelated to South Asian history

Game	Year, origin	Players	Core mechanic	Gap 1757 addresses
Among Us	2018, Innersloth	4–15	Crewmates do tasks, Impostors kill and sabotage; meetings and votes	Action-based movement rather than structured team votes; peaked at about 3.8 million concurrent players in 2020

What 1757 keeps from prior work:

- From The Resistance: Leader nomination, team vote, secret mission cards, five-rejection deadlock, and first side to three.
- From Werewolf and Town of Salem: a large cast of roles with special abilities and a neutral role.
- From online versions: the server as moderator.

How 1757 differs:

1. **Real history as the role set.** All 15 named roles are people from the Battle of Plassey on 23 June 1757. Their abilities follow their historical parts: Mir Jafar the betrayer, loyal Mir Madan, the bankers of Jagat Seth, and the conspirators Rai Durlabh and Yar Lutuf. Saint Frais follows de St. Frais, whose French gunners fought for the Nawab.
2. **Graded and deceptive information.** The game uses decoys (Tara Bai, Ghasheti Begum), an invisible role (Rai Durlabh), a time-limited glimpse (Mir Madan sees the Reds only in Round 1) and secret double votes. The Resistance's roles mostly know everything or nothing.
3. **A two-stage endgame.** Mir Jafar can overturn a Green win, and then Jagat Seth can steal the whole game.
4. **No elimination.** Every player stays in until the end.
5. **Automated and cheat-resistant.** Each browser receives only its own role's view.
6. **Host customisation.** Royal Decree mode lets the host set per-round timers, pick allowed roles and assign seats.

Technical sources consulted: the official documentation of Socket.IO (rooms, acknowledgements, reconnection), Fastify, Prisma, Better Auth, Next.js, Expo, WebRTC (peer-to-peer media and signalling), Zod and Vitest. Our engine also follows the reducer

pattern used by state-management libraries such as Redux: a pure function from (state, action) to the new state.

CO2 evidence. Our SRS (`/doc/srs.pdf`) is a requirements workbook: a project summary plus the 19 functional requirements and their 30 user stories. It has no separate section on existing games, so this section is the written record of that comparison. The differences listed above are what FR_4 (secret roles), FR_5 (faction vision), FR_8 (abilities) and FR_18 (server-side validation) ask for.

4. Requirements (SRS Session)

The SRS (`/doc/srs.pdf`), presented on 16 July 2026, is a requirements workbook. It gives a project summary and 19 functional requirements (FR_1–FR_19), refined into 30 user stories (US_1–US_30). Each story has acceptance criteria, a priority (13 High, 13 Medium, 4 Low) and a status.

Stakeholders

Stakeholder	Interest
Players (registered and guests)	Fair, fast, fun matches; their secret role stays secret; progress is saved
Lobby hosts	Control over room privacy, size, timers, roles and membership
Administrators	Visibility of system health, game balance and abusive users
Course supervisors and examiners	A complete, well-engineered system that meets SWE 4404 outcomes
Development team	A codebase that is testable, maintainable and deployable on free-tier infrastructure

Functional requirements and traceability

FR	Requirement (the system shall…)	User stories	Stakeholder need
FR_1	Support real-time lobby creation and customization	US_1, US_2	Hosts invite friends and tune the rules
FR_2	Manage matchmaking and room joining	US_3	Players find a game without an invite
FR_3	Support real-time public communication	US_4, US_18	Players debate and accuse
FR_4	Distribute secret roles and adapt the interface	US_5	Players know their goal
FR_5	Reveal faction-specific information to authorized roles	US_6, US_7, US_20	Hidden information drives deduction
FR_6	Moderate a phase-driven discussion loop	US_8	Players know how long they have
FR_7	Run a stateful voting engine for proposed teams	US_9, US_11, US_19	Players control who goes on missions
FR_8	Enable active abilities for special roles	US_10, US_21, US_23	Roles feel distinct
FR_9	Resolve secret mission cards	US_12	Traitors can sabotage without being exposed
FR_10	Let hosts moderate lobbies	US_13	Hosts remove disruptive players
FR_11	Support secure registration, login and sessions	US_14, US_15	Players keep an identity and progress
FR_12	Maintain profiles, avatars and match statistics	US_16, US_28	Players track performance
FR_13	Manage leader rotation and minister nominations	US_17	The leader picks a team
FR_14	Compile results and evaluate victory conditions	US_22, US_24	Everyone knows who won and why

FR	Requirement (the system shall…)	User stories	Stakeholder need
FR_15	Show a post-match breakdown and update profiles	US_25	Players review strategy after the game
FR_16	Deliver immersive audio-visual content	US_27	Tension and atmosphere
FR_17	Handle disconnections and reconnections	US_29	A dropped connection does not ruin a match
FR_18	Enforce game rules on the server	US_30	Nobody can cheat by tampering with the client
FR_19	Handle lobby departures and host migration	US_26	A lobby survives its host leaving

Non-functional requirements

Category	Requirement	Stakeholder need
Security	Full game state never leaves the server; each player receives only their own view; every input is validated; identity comes from the session	Fair play, no cheating
Security	Rate limits: 100 HTTP requests/min per IP, 10 game actions/s, 5 chat messages per 5 s	Protection from spam and abuse
Reliability	Game state is saved to Redis after every change; a player who drops has a 30 s grace window, after which the game acts for them	Matches finish even with bad connections
Performance	State changes reach all players in real time over WebSockets	Smooth live play
Usability	Plays in a desktop or mobile browser without installing anything; guest play needs no sign-up	Low barrier to entry

Category	Requirement	Stakeholder need
Maintainability	Strict TypeScript; all game rules in one pure package; tunable values in one config file; automated tests in CI	The team can change rules safely
Portability	One engine shared by the server and the web client's types	Same rules on every platform

The SRS lists functional requirements only. These quality goals come from its project summary (real-time play for 6 to 15 players under a strict state machine) and from the design report (three tiers, Redis for fast live state).

Context and sustainability

- **Cultural and educational value (SDG 4, Quality Education).** The game teaches a key event in Bengal's history through play.
- **Low resource use and cost.** The system runs on free tiers (Render, Vercel, Neon PostgreSQL, Upstash Redis). Voice chat is peer-to-peer, so audio does not pass through or load our server.
- **Inclusion.** Guest play needs no email address. The game runs in a browser on modest devices, and rules are explained in plain language in-game.
- **Data minimisation.** Only account data and match history are stored; guests leave only an anonymous ID.

Feedback received and changes made

Source	Feedback	What we changed
Evaluation panel	Concern that the scope was too large to finish in one semester	Fixed v1 to a complete playable match (plan items 1–20) and moved extras to post-mid; the full game was delivered and demonstrated on 28 Oct 2026
Evaluation panel	Why use Redis instead of keeping game state in server memory?	Kept Redis, with the reasoning in Section 5

Source	Feedback	What we changed
Supervisor (Samia Nawsheen)	Add background music and a custom room mode	Added three looping background tracks, and Royal Decree mode (per-round timers, allowed roles, host-assigned seats, pause and skip)
Advisor	A social deduction game needs voice, not just text	Added peer-to-peer WebRTC voice chat

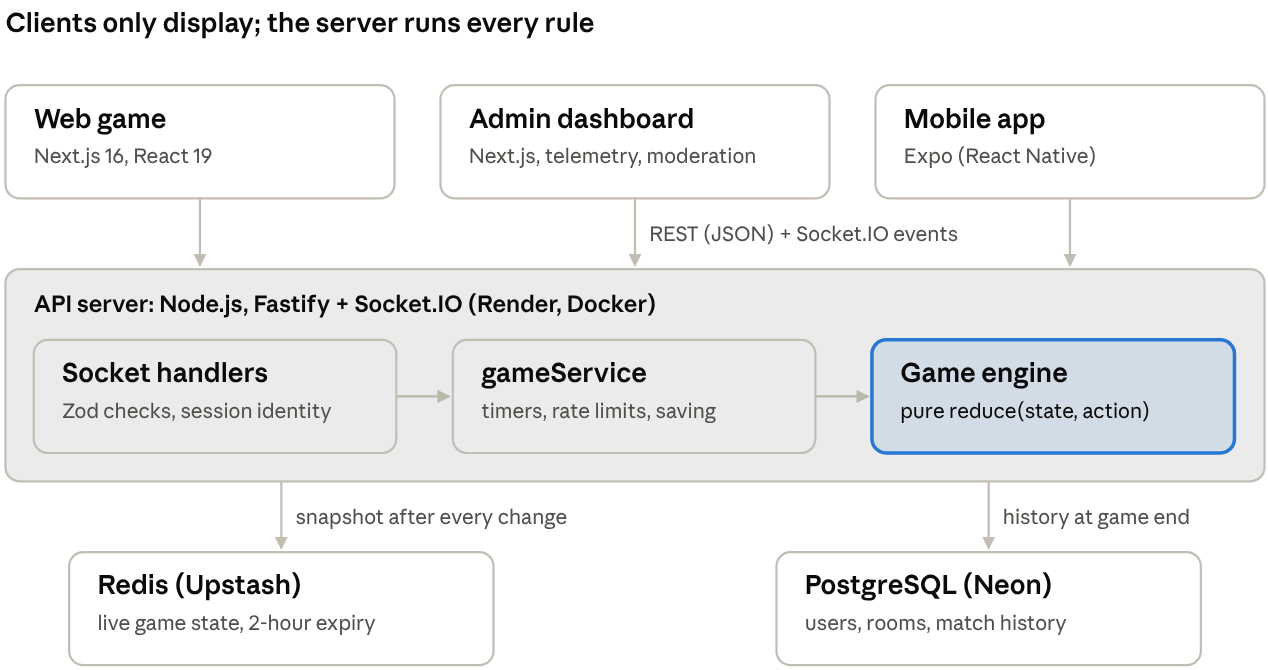
No other changes were requested.

We also changed some requirements ourselves, while reconciling the requirements spreadsheet with the team's game design document:

Story	Original	Changed to	Reason
US_6	Siraj sees all EIC spies	Dropped; Siraj sees Lutfunnesa, with Tara Bai as a decoy	Seeing every spy made Green too strong
US_10	de Bussy peeks at a minister's card	Once per game, de Bussy and a target see each other's exact role	Matches the game design document
US_21	Jean Law locks a Red player to deny win credit	Jean Law locks any player; a locked Green flips to winning with Red	Matches the game design document
US_23	St. Frais (Red) peeks at a mission card	Saint Frais is Green, and his ballot vote counts twice	Historically de St. Frais fought for the Nawab
US_29	60 s reconnect grace	30 s grace, then the game acts for the player	Keeps matches moving

5. System Design (Design Session)

We chose a three-tier, server-authoritative design. Clients only display what the server sends. One Node.js server runs a pure game engine, and data is split between Redis for live games and PostgreSQL for permanent records. The full design document is in `/doc/design.pdf`.



1757 architecture · 3 clients, 1 server, 2 data stores

Every client talks to one API server. Inside it, the socket handlers validate messages and `gameService` calls the pure game engine. Live state is snapshotted to Redis after every change, and finished matches go to PostgreSQL. Voice audio travels directly between browsers (WebRTC); the server only relays the connection setup.

Components and their trace to the SRS

Component	Responsibility	Traces to
Web client (Next.js)	Lobby, game table, chat, voice, profile, post-game report	FR_1–FR_4, FR_6, FR_12, FR_15, FR_16
Mobile client (Expo)	Sign-in and global chat on phones	FR_11
Admin dashboard (Next.js)	Telemetry (active rooms, users, faction win rates), match logs, user moderation	Admin stakeholder needs
API server (Fastify + Socket.IO)	Authentication, room REST endpoints, real-time events, validation, rate limits, timers, persistence	FR_1, FR_2, FR_3, FR_10, FR_11, FR_17, FR_18, FR_19

Component	Responsibility	Traces to
Game engine (pure TypeScript package)	All rules: phases, voting, missions, roles, vision, abilities, endgame, scoring	FR_4–FR_9, FR_13, FR_14, FR_18
Redis	Live game snapshot after every change (2-hour expiry)	FR_17, reliability NFR
PostgreSQL (Neon) via Prisma	Users, sessions, rooms, and finished match history	FR_11, FR_12, FR_15

Data model and database schema

The Prisma schema has three groups of tables.

Group	Model	Key fields	Relations
Accounts (Better Auth)	User	id, name, email (unique), username (unique, optional), isAnonymous, avatarId, xp (default 0), level (default 1)	has many Session, Account, Room, MatchPlayer
Accounts	Session	token (unique), expiresAt, userId	belongs to User (cascade delete)
Accounts	Account	providerId, accountId, access/refresh tokens, password hash	belongs to User (cascade delete)
Accounts	Verification	identifier, value, expiresAt	none
Lobby	Room	code (unique), hostUserId, status (WAITING, IN_PROGRESS, FINISHED), mode (STANDARD, CUSTOM), maxPlayers, discussionSeconds, roundDiscussionSeconds[], enabledRoles[], bannedUserIds[]	has many RoomPlayer
Lobby	RoomPlayer	roomId, userId, seat, role, team	unique (roomId, userId)

Group	Model	Key fields	Relations
Match history	Match	roomCode, playerCount, winner (GREEN, RED, GOLD), reason (ROUNDS, MIR_JAFAR_GUESS, JAGAT_SETH_GUESS), roundsGreen, roundsRed, config (JSON)	has many MatchPlayer, MatchRound
Match history	MatchPlayer	seat, role, team, won, userId (null for guests), guestId	belongs to Match; unique (matchId, seat)
Match history	MatchRound	number, teamSeats[], approved, successCount, sabotageCount, result, reason (MISSION, DEADLOCK)	belongs to Match; has many BallotVote
Match history	BallotVote	attempt, seat, vote (APPROVE, REJECT), weight	belongs to MatchRound; unique (roundId, attempt, seat)

Live game state is not stored in PostgreSQL. Match history is written in one go when a match ends, which keeps database traffic tiny during play.

Design alternatives compared

Decision	Option A	Option B	Chosen and why
Who runs the game	Host's browser runs the rules (peer-hosted)	Server runs the rules (server-authoritative)	**B.** A browser that holds all the roles lets the host cheat, and the game dies if the host leaves. Costs one server, but it is the only way to keep secrets fair.
Real-time transport	HTTP polling every few seconds	WebSockets via Socket.IO	**B.** Polling adds delay and wasted requests for 6–15 players. Socket.IO adds rooms, acknowledgements and automatic reconnection.

Decision	Option A	Option B	Chosen and why
Engine style	Classes with mutable state (a Game object with methods)	Pure reducer `(state, action) → { state, events }`	**B.** No hidden state, so a match can be replayed from its actions, unit-tested without a server, and resumed from a snapshot.
Where live state lives	Write every action to PostgreSQL	In memory, snapshot to Redis, history to PostgreSQL at the end	**B.** Hundreds of writes per match on a free database tier would be slow and costly. Redis snapshots still survive a restart.
Voice chat	Relay audio through our server	Peer-to-peer WebRTC, server only relays setup messages	**B.** No audio bandwidth on our free-tier server; lower latency. Trade-off: a large room means each browser holds many connections.

Against stakeholder needs, the chosen design gives:

- **Fairness:** the authoritative server and per-player views.
- **Usability:** instant updates and reconnection.
- **Scalability:** a single instance today; a Socket.IO Redis adapter is the planned path to more.
- **Resource use and lifecycle cost:** everything runs on free tiers.

Feedback from the Design session and changes made

The panel asked: **why use Redis instead of keeping game state in the server's memory?** We kept Redis, for four reasons.

1. **Speed still comes from memory.** The live game runs in the server's memory. Redis only holds a copy, written after each change, so play is never slowed by it.
2. **Surviving restarts and redeploys.** Render restarts the server on every deploy and the free tier sleeps when idle. With memory alone, every running match would vanish. On boot, the server restores live games from Redis.
3. **Reconnecting players.** A player who drops and returns within the 30-second grace window gets their view rebuilt from the saved state.

4. **A path to scaling.** Redis is what a Socket.IO Redis adapter needs to run several server instances later.

The cost is one extra free-tier service. For local development, the server falls back to an in-memory store when Redis is unavailable.

From the supervisor and advisor, the design gained two components after the Design session: a voice-signalling relay for peer-to-peer voice chat, and a per-room custom-settings model for Royal Decree mode.

6. Implementation (Implementation Session)

The code base is about 11,000 lines of TypeScript source plus about 5,000 lines of tests, organised as a Turborepo monorepo:

- **4 deployable apps** under `apps/`;
- **8 shared packages** under `packages/`;
- **all game rules** in one pure package, `packages/game-engine`.

6a. Project structure

`apps/` holds what we deploy and `packages/` holds code the apps share. Every rule lives in `game-engine`, which does no input/output, so the server and the web client run identical rules and the engine's tests need no server. Inside the API, folders are split by job:

- `socket/` wires events;
- `realtime/` runs live games, timers and the Redis snapshot store;
- `lobby/` handles rooms;
- `persistence/` records finished matches.

Inside the engine, folders are split by phase, role, ability and vision, so a new role touches one folder.

```
1757-RPG/
├── README.md                how to build, run and test the project
├── doc/                     srs.pdf, design.pdf, final-presentation.pdf,
│                            final-report.pdf, screenshots/
├── .github/workflows/ci.yml
├── render.yaml
└── code/                    all source code (tests, build output,
```

```
node_modules
    |                        and the generated Prisma client excluded)
    ├── apps/
    |   ├── api/                Fastify + Socket.IO game server
    |   |   └── src/
    |   |       ├── index.ts        server entry, /health, graceful shutdown
    |   |       ├── auth-routes.ts  Better Auth routes
    |   |       ├── admin/          admin telemetry API
    |   |       ├── lib/            session, Redis client
    |   |       ├── lobby/          room codes, rules, lobby views
    |   |       ├── persistence/    match recorder (writes at POST_GAME)
    |   |       ├── realtime/       gameService, scheduler, snapshotStore,
    |   |       |                   voiceRooms, rateLimit, schemas
    |   |       ├── routes/rooms.ts room REST endpoints
    |   |       ├── socket/index.ts Socket.IO event wiring
    |   |       └── users/          profiles
    |   ├── web/                Next.js 16 game client
    |   |   ├── app/            landing, login, signup, home, room/[id]
    |   |   ├── components/     ActionPanel, VoiceChatPanel, ...
    |   |   └── lib/            audio, music, useVoiceChat, game types
    |   ├── admin/              Next.js admin dashboard
    |   └── mobile/             Expo (React Native) app
    ├── packages/
    |   ├── game-engine/        pure TypeScript rules engine (the FSM)
    |   |   ├── src/
    |   |   |   ├── reduce.ts       reduce(state, action) → { state, events }
    |   |   |   ├── createGame.ts   deals roles, sets up a match
    |   |   |   ├── rng.ts          seeded random numbers
    |   |   |   ├── config/         team mixes, timers, weights
    |   |   |   ├── phases/         one module per game phase
    |   |   |   ├── roles/          role catalog
    |   |   |   ├── abilities/      deBussy, jeanLaw
    |   |   |   └── vision/         viewFor (per-player view)
    |   |   └── test/           23 Vitest test files
    |   ├── db/                 Prisma schema and client
    |   ├── auth/               Better Auth setup
    |   └── types/  config/  ui/  eslint-config/  typescript-config/
    ├── docs/                   event protocol, REST API and telemetry docs
    └── Dockerfile  docker-compose.yml  turbo.json
```

6b. Communication between components

There are three conversations: front end ↔ back end, back end ↔ engine, and back end ↔ data.

Front end ↔ back end. Ordinary requests (sign-in, rooms, profiles) are REST calls to Fastify with JSON bodies. The live game uses Socket.IO. Every client event receives an acknowledgement, either `{ ok: true }` or `{ ok: false, error: { code, message } }`.

Direction	Events	Payload	Channel
client → server (13)	`room:join`, `room:leave`	`{ roomId }`	—
	`game:start`, `game:sync`, `game:skip`, `game:pause`, `game:resume`	`{ roomId }`	—
	`game:action`	`{ roomId, action }` (6 action types, below)	—
	`chat:send-message`	`{ roomId, … }`	—
	`voice:join`, `voice:leave`, `voice:mute`, `voice:signal`	`{ roomId, … }` (WebRTC offer, answer or candidate)	—
server → client (18)	`game:view`	`{ PlayerView }`: me, vision, reveals, myCard, pending, game	private, `user:<id>`
	`game:event`	`{ roomId, events }`, e.g. PHASE_CHANGED, MISSION_RESOLVED, GAME_ENDED	room
	`game:started`, `game:error`, `game:join`, `game:leave`	status data; errors go only to the sender	room or one player
	`chat:history`, `chat:message`, `player:status`	chat and presence	room

Direction	Events	Payload	Channel
	`room:online-list`, `room:update`, `room:settings`, `room:kicked`, `room:player-kicked`, `lobby:list`	lobby state	room or lobby
	`voice:participants`, `voice:peers`, `voice:signal`	WebRTC signalling	room or one peer

The API also exposes 24 REST endpoints: health, auth, 10 room routes, 6 user routes and 5 admin routes.

The `action` inside `game:action` is a Zod discriminated union of six data classes:

```
const clientAction = z.discriminatedUnion("type", [
  z.object({ type: z.literal("NOMINATE"), ministers: … }),
  z.object({ type: z.literal("CAST_VOTE"), vote: z.enum(["APPROVE",
"REJECT"]) }),
  z.object({ type: z.literal("PLAY_CARD"), card: … }),
  z.object({ type: z.literal("DEBUSSY_REVEAL"), targetId: id }),
  z.object({ type: z.literal("JEANLAW_LOCK"), targetId: id }),
  z.object({ type: z.literal("GUESS"), targetId: id }),
]);

// the acting player always comes from the session, never from the payload
function toEngineAction(action: ClientAction, playerId: string): Action {
  return { ...action, playerId };
}
```

Back end ↔ engine. The server calls `reduce(state, action)` and receives the new state plus a list of events. Both are plain TypeScript objects.

Back end ↔ data.

- **Redis** holds the whole game state as one snapshot under `game:<roomId>`, kept for 2 hours, rewritten after every change.
- **PostgreSQL**, through Prisma, holds the models in Section 5. When a match ends, the match recorder writes one `Match` row and its `MatchPlayer`, `MatchRound` and `BallotVote` rows together.

End-to-end example: one ballot vote.

1. The player clicks Approve. The browser emits `game:action { roomId, action: { type: "CAST_VOTE", vote: "APPROVE" } }`.
2. The socket handler validates the payload with Zod, and `toEngineAction()` attaches the voter's `playerId` from the session.
3. `gameService` runs `reduce()` on the live game. The ballot phase records the vote with its weight (2 for Saint Frais or Yar Lutuf).
4. The new state is saved to Redis, and the voter's acknowledgement returns `{ ok: true }`.
5. Every player receives a fresh `game:view`, and the room receives a `game:event`. The public tally shows only the headcount, so a double vote stays secret.
6. When the match ends, the match recorder saves this vote in PostgreSQL as a `BallotVote` row (`attempt`, `seat`, `vote`, `weight`).

6c. Division of classes and methods

We chose functions over classes on purpose. The code has only 7 classes in about 9,550 lines, and each one is used where an object must hold state over time. Everything else is plain functions or React function components.

- **Single responsibility:** one module per game phase; roles as data in one catalog; abilities and vision in their own modules; on the server, `socket`, `gameService`, `scheduler`, `snapshotStore` and `roomRepository` each do one job.
- **Pure core, input/output at the edges:** only `apps/api` touches sockets, Redis and PostgreSQL.
- **Layering:** dependencies point one way, from apps to packages, never back.
- **Reuse and extension:** a new role is a catalog entry plus a vision rule, with no change to the state machine.

Sizes were measured with the TypeScript compiler over all non-test source, skipping generated Prisma code and `.d.ts` files.

Unit	Count	Average lines	Median	Maximum
Methods and functions	363	22.7	8	930 (`RoomPage`)
Classes	7	111.7	—	345 (`GameService`)

Unit	Count	Average lines	Median	Maximum
Modules (files)	102	93.6	—	899 (`room/[id]/page.tsx`)
Game-engine methods only	71	13.0	—	78

Justification.

- The 930-line maximum is `RoomPage`, the in-game React screen. A React component's whole page markup counts as one function, and this one is the single real outlier and the first thing to split into smaller components.
- The next largest are also React pages (`HomePage` 439, `CustomRoomSettings` 285) and two registration functions (`registerRoomRoutes` 230, `registerSocketHandlers` 205), which are mostly lists of routes and event handlers.
- `GameService` (345 lines, 23 methods) is the one class that owns all live matches and their timers.
- The engine, which holds the logic, averages 13 lines per function.

Implemented features

- Guest play (themed names such as "Swift Sepoy 42") and email/password accounts (Better Auth)
- Public and private rooms with a 6-character code; host settings for player cap (6–15), discussion timer (30–300 s), visibility and allowed roles; host migration; kick with ban (API)
- Royal Decree custom mode: per-round timers, allowed characters, host-assigned seats, pause, resume and skip
- The full match flow: role reveal, discussion, selection, weighted ballot, mission, round result, endgame guesses, post-game report
- 15 historical roles plus 2 fillers, with decoys, an invisible role, a time-limited glimpse, weighted votes and two once-per-game abilities
- Room text chat, a ballot history panel (Chronicle) and peer-to-peer voice chat
- 30-second reconnect grace with automatic actions after it; live games restored from Redis after a restart
- XP and levels, win rate and stats

- Pixel-art historical styling, 32 colour themes, 8-bit sound effects and three background music tracks
- Admin dashboard: telemetry (users, matches, faction win rates), searchable match log, user directory, active rooms
- Rate limits, Zod input validation, health check and graceful shutdown

Refactoring before the Implementation session

The largest refactor replaced the original game engine. In the first version (June–July), the game ran as asynchronous socket handlers that wrote state straight into PostgreSQL and Redis on every action. A Phase 0 audit of the code found six problems:

- no server-side timers;
- the room routes trusted a client-sent `userId`, so anyone could act as any player;
- roles leaked to other players (a broadcast Jean Law lock, Mohon Lal's suspects carrying their teams, array order revealing Siraj's partner);
- three rejections ended the whole game;
- wrong role compositions at 14–15 players;
- Postgres traffic on every move.

We rebuilt the rules as the pure `game-engine` package (modules M1–M4) and deleted the legacy engine.

Date	Author	Commit	Change	Why
2026-07-30	Safwan	dc23e84	Reorganised the first engine into `game/engine.ts` and `socket/index.ts` (+2,757 / −1,688 lines)	Consolidate scattered round and room handlers
2026-10-26	Wasi	05af342	Deleted the legacy `apps/api/src/game/*` (−1,441 lines)	Replaced by the authoritative game service and pure engine
2026-10-26	Wasi	7d66f68	Rebuilt the room routes on session identity	Old routes trusted a client-sent userId
2026-10-27	Saif	ced1352	Moved the profile route into the users module	Single responsibility (module M8)

Date	Author	Commit	Change	Why
2026-10-27	Wasi	401ba09	Web client uses the engine's own view types (−160 lines)	One source of truth for `game:view`
2026-10-27	Wasi	05edf05	Stopped tracking build files and a generated code graph (−7,022 lines)	Repository hygiene
2026-10-27	Wasi	d363498	Replaced the CI Vercel job with Vercel's Git integration	Hobby plan blocked deploys by non-owners and the job hung

Old database columns (`RoundRecord` , `RoomPlayer.isSelector` , `isLockedByJeanLaw` , `hasUsedDeBussyLock`) were left in the schema under an "additive-only" migration policy. The API no longer reads them.

CO1 and CO3 evidence

CO1, concepts in the code:

- The finite state machine lives in `reduce.ts` and `phases/` .
- Seeded randomness is in `rng.ts` .
- Per-player projection is `vision/viewFor` .
- Weighted tallies are in the ballot phase.
- Relational modelling, constraints and cascades are in `schema.prisma` .
- Discriminated unions validate actions in `schemas.ts` .

CO3, follows the design: the implementation matches the Section 5 architecture: server-authoritative, pure engine, Redis snapshot, PostgreSQL at match end.

Justified departures from the design:

1. **Legacy columns remain** because of the additive-only migration policy. `Room` status and score are still written at game start and finish, so PostgreSQL is touched at those two points, not only at match end.
2. **Redis is optional.** If it is unreachable, the API falls back to an in-memory store with a warning, so local development works without Redis. In that mode, games do not survive a restart.
3. **One API instance.** Chat history, timers, disconnect tracking and voice rooms live in process memory, and there is no Socket.IO Redis adapter yet.

4. **Cross-origin hosting workarounds.** Hosting the web app on Vercel and the API on Render led us to trust `*.vercel.app` origins, proxy API calls through Next.js rewrites, and turn off Better Auth's origin and CSRF checks. These are deployment workarounds to revisit (Section 8).
5. **Fastify instead of Express.** The design report names an Express server. We built the API on Fastify (reasons in Section 7); the routes and Socket.IO events are the same.
6. **Plain data and pure functions instead of classes.** The class diagram has `Match`, `Round`, `Player` and an abstract `Role` with subclasses. The engine keeps these as typed data and pure functions (`reduce`, `viewFor`), so a whole game is one JSON object that is easy to test and to snapshot to Redis.
7. **Votes are counted by the engine, not in Redis.** In the design's sequence diagram, Redis gathers the ballots. In the build, the engine counts them in memory and Redis stores a snapshot after each action, which is simpler and keeps every rule in one place.
8. **Web first, mobile second.** The SRS and design named a React Native client. Build errors made us ship the Next.js web app as the main client and keep mobile to sign-in and chat (Section 9).
9. **Database tables.** The design's `USER`, `MATCH` and `MATCH_HISTORY` became `User`, `Match` and `MatchPlayer`; Better Auth added its own session and account tables, and `Room` holds lobby settings.

7. Tools, Version Control, Testing, and CI/CD

The project uses one TypeScript toolchain end to end, git with one branch per module, 419 automated tests run in CI, and a pipeline that deploys the API to Render and the web app to Vercel from `main`.

Frameworks and tools, against an alternative

Need	Chosen	Alternative considered	Why chosen	Limitation
Language	TypeScript (strict)	Plain JavaScript	One type system across server, engine and clients catches mismatched messages at compile time	Build step; type gymnastics for some library types

Need	Chosen	Alternative considered	Why chosen	Limitation
Monorepo	pnpm workspaces + Turborepo	Separate repositories	Shared engine and types without publishing packages; cached builds	One repo holds every app, so CI and deploy paths need care
API server	Fastify	Express	Faster, schema-friendly, first-class plugins (helmet, CORS, rate limit)	Smaller ecosystem than Express
Real-time	Socket.IO	Raw WebSocket (`ws`)	Rooms, acknowledgements, automatic reconnection and fallbacks built in	Its own protocol; clients must use the Socket.IO client
Validation	Zod	Hand-written checks	One schema gives runtime checks and TypeScript types	Adds a dependency to every boundary
Database access	Prisma (v7) on Neon PostgreSQL	Raw SQL / TypeORM	Typed queries, migrations, readable schema	Generated client; complex queries can be awkward
Live state	Redis (ioredis), in-memory fallback	Keeping state only in memory	Survives restarts; ready for multi-instance scaling	Another service to run; Upstash free-tier limits
Authentication	Better Auth	Hand-rolled JWT	Sessions, email/password accounts and anonymous guests out of the box	Its tables shape our User model

Need	Chosen	Alternative considered	Why chosen	Limitation
Web	Next.js 16, React 19, Tailwind 4 + daisyUI	Plain React (Vite)	Routing, production builds and Vercel deployment with no setup	Server/client component rules add complexity
Mobile	Expo (React Native)	Native Android/iOS	Reuses TypeScript and React skills	Still an early prototype
Voice	WebRTC (peer-to-peer mesh)	A media server (SFU)	No audio through our server, no cost	Mesh grows heavy as rooms get large
Testing	Vitest	Jest	Fast, native TypeScript and ES modules	Fewer plugins than Jest
Deployment	Docker on Render (API), Vercel (web, admin)	One VPS	Free tiers, automatic deploys from git	Render free tier sleeps when idle, so the first request is slow

Version control and workflow

The repository is `SaifSarker11/1757-RPG` on GitHub (private), with 22 remote branches and 218 non-merge commits from 19 May to 28 October 2026.

- **Agreed rules:** pull before starting, branch from `dev`, merge into `main` through pull requests, never push directly to `main`, never commit secrets.
- **One branch per module:** `feat/m1-game-engine` to `feat/m11-deployment-prod`, plus `fix/…` branches.
- **Engineering log:** each session logged in `PROGRESS.md`; ownership, tasks and decisions in the team workbook.

Member	Commits (main)	Lines + / − (main)	Merged PRs	Active
Wasi Omar Syed	109	+13,889 / −9,497	4 (#6, #7, #8, #9)	Jun–Oct 2026
Md. Saif-Al Sarker	82	+11,764 / −553	1 (#2)	May–Oct 2026
M Safwan Hasan Khan	10	+22,734 / −5,085	3 (#1, #3, #4)	May–Jul 2026

Line counts are skewed by generated files: Safwan's include about 7,000 lines of a generated code graph that Wasi later removed. Without the lockfile, the all-branch figures are +15,960 (Wasi), +5,154 (Saif) and +18,550 (Safwan).

What went less well, honestly. Only 8 pull requests were merged; most module branches were merged without a pull request, and some fixes were pushed straight to `main`. `main` has no branch protection, so a passing CI run was not enforced: two of the last five runs on `main` failed before being fixed. Adding a protection rule that requires the `CI/CD / ci` check is our first process fix.

Testing

All 419 tests pass (3 marked to-do), in 42 test files across the engine and the API.

Package	Test files	Passed	Failed	To-do	Duration
`packages/game-engine`	23	253	0	3	41.8 s
`apps/api`	19	166	0	0	3.4 s
Total	**42**	**419**	**0**	**3**	

Level	What it checks
Unit (engine)	Ballot tally, tie rule and weights; mission card rules and thresholds; nomination and leader rotation; every role's vision; de Bussy and Jean Law; endgame guesses; scoring; phase timers; settings validation; seeded RNG

Level	What it checks
Property-based	Role dealing over 300 seeds × 10 player counts (6–15); full timeout-driven matches at every player count; Tara Bai over 100 seeds
Security (leak)	`viewFor` checked against an independent oracle over 2,000 game states (10 player counts × 40 seeds × 5 phases), for every viewer; ability reveals reach only the players involved (200 seeds at 15 players)
Unit (API)	Room rules, room codes, schemas, match recording, disconnect grace, game service, lobby cleanup, rate limiter, socket auth, storage, voice rooms, users and XP, admin routes
Integration	An in-process Socket.IO test, end to end: authentication, private errors, rate limits and per-player views
Manual	Six-player play-tests in separate browser windows, following `docs/manual-testing.md`, and the live demo

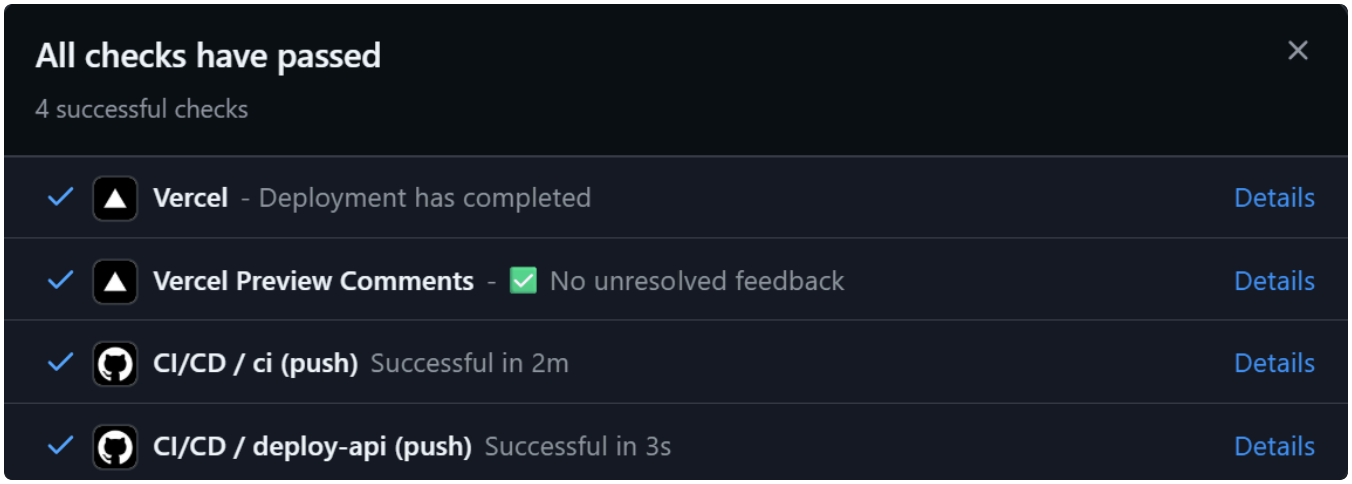
Limitations: coverage reporting is not configured; the web, admin and mobile apps have no automated tests; API tests mock the database rather than using a real PostgreSQL or Redis.

CI/CD pipeline

GitHub Actions (`.github/workflows/ci.yml`, workflow "CI/CD") runs on every push and pull request to `main`. It has two jobs:

- **`ci`:** install with a frozen lockfile → Prisma generate → typecheck → test → build. It takes about 2 minutes.
- **`deploy-api`:** after `ci` passes on `main`, it triggers a Render deploy through the Render API.
- **Web app:** deploys through Vercel's Git integration, which also posts preview deployments on pull requests.
- **Missing:** a lint step.

The API is built as a multi-stage Docker image that runs as a non-root user and is deployed to Render from `render.yaml`. Its `/health` endpoint checks the database and Redis, and the server shuts down gracefully. The web and admin apps deploy automatically on Vercel.



A passing run on `main`: the Vercel deployment and the two CI/CD jobs all succeeded.

8. Professional, Ethical, and Legal Considerations

The libraries we use are under permissive open-source licences. We store little personal data, protect it with standard security measures, and present the history respectfully. One lesson learned: database and service credentials had been pasted into the team's shared planning document, so we rotated those credentials and now keep secrets only in environment variables.

Licences

Our 783 production dependencies are almost all under permissive licences (from `pnpm licenses list --prod`):

Licence	Packages
MIT	760
ISC	45
Apache-2.0	41
BSD-3-Clause	18

Licence	Packages
BlueOak-1.0.0	9
BSD-2-Clause	8
Others (Unlicense, 0BSD, MIT/CC0, MPL-2.0, CC-BY-4.0, Python-2.0, dual licences)	13

Packages worth noting, none of which stops academic or commercial use:

- `sharp`'s Windows binary is Apache-2.0 plus LGPL-3.0 (libvips), used unmodified as a dynamically linked library by Next.js.
- `node-forge` is BSD-3-Clause or GPL-2.0, and we use it under BSD.
- `lightningcss` is MPL-2.0 (file-level copyleft, unmodified).
- `caniuse-lite` is CC-BY-4.0 (data).

There is no AGPL and no unknown licence.

Asset	Source	Licence
Fonts: Cinzel, Press Start 2P, VT323, Geist, Geist Mono	Google Fonts	SIL Open Font License 1.1
Music: "Relax Ay Sitar", "Sitar and simple beats", "The fury of Sitar" by Antti Luode	Wikimedia Commons, unmodified	CC BY 3.0, credited in `CREDITS.md` and the in-game music menu
Sound effects	Generated in code (Web Audio API)	Our own
Favicon and starter SVGs	create-next-app template	Template defaults; to be replaced

Personal data and privacy

Who	What is stored
Registered users	Name, email, username, avatar ID, XP, level; a password hash (scrypt, never the password); sessions with IP address and browser user agent; match results (seat, role, team, win)

Who	What is stored
Guests	A user row marked anonymous, with a generated name and a placeholder email; sessions with IP address and user agent; match results
Nobody	Chat messages and voice: chat is kept only in server memory, and voice never passes through our server

- **Purpose:** running games, showing profiles and stats, and game-balance figures for the admin dashboard. Nothing is sold or shared.
- **Retention:** live game snapshots expire after 2 hours. Sessions last 7 days and are refreshed daily (Better Auth defaults).
- **Gap:** there is no account deletion or data export yet; users must ask the team. This is future work.
- **Legal context:** we follow general data-protection principles: consent at sign-up, data minimisation, security and purpose limitation.

Security measures

Threat	Measure
Cheating by reading other players' roles	The full state never leaves the server; each player gets only `viewFor(state, player)`; leak tests over 2,000 game states in CI
Acting as another player	Player identity always taken from the session, never from the message (fixed in the engine rebuild)
Malformed or malicious input	Zod schemas on every socket payload and REST body; illegal moves rejected with an error code sent only to the sender
Spam and denial of service	100 HTTP requests per minute per IP; per-socket token buckets: 5 chat messages per 5 s, 10 game actions per s, 60 voice-signal messages per s

Threat	Measure
Web attacks	Helmet security headers; CORS limited to our web and admin origins; cookies `httpOnly`, and `secure` with `sameSite=none` in production; HTTPS everywhere
Password theft	Passwords hashed with scrypt by Better Auth
Leaked credentials	Secrets only in Render and Vercel environment variables; `.env` git-ignored; a scan of the git history found no real tokens or passwords
Container risks	Multi-stage Docker image running as a non-root user

Known weaknesses to fix. An honest audit found three gaps:

1. To make cross-origin login work between Vercel and Render, Better Auth's origin and CSRF checks were turned off and any `*.vercel.app` origin is trusted.
2. `GET /api/users/:userId/stats` needs no login.
3. A development-only route for creating users is active whenever `NODE_ENV` is not `production`.

The fixes:

- serve the web app and API from one domain, or list exact origins, and turn the CSRF and origin checks back on;
- make sure production always sets `NODE_ENV=production`.

Ethical and legal issues

- **Historical sensitivity.** The Battle of Plassey is a painful moment in Bengal's history. Roles are named after real people, and in-game text describes their historical parts factually without mocking any community or faith.
- **Fair play.** The server enforces every rule, so no player can gain an unfair advantage by modifying the client.
- **Healthy play.** No in-game purchases, gambling or advertising.
- **Moderation.** Hosts can kick and ban, and admins can moderate users, which limits abusive behaviour.

- **Professional conduct.** Work was shared equally, credited in the README, and tracked in version control.

9. Project Management and Feasibility

We planned in two waves: a playable core first (task items 1–20), then polish and extras (items 21–41). We tracked every task with an owner and status in a shared team document, logged each work session in `PROGRESS.md`, and reprioritised at the mid-progress review.

Timeline: planned vs actual

Milestone	Planned	Actual (from git)	Evidence
Monorepo scaffold and CI	Start of term	2026-05-19	a7babb1 (Saif)
Login, sign-up, guests	v1 core (items 1–20)	2026-05-20	PR #1, 3f9267a (Safwan)
Real-time chat	v1 core (items 1–20)	2026-05-21	PR #3, 5f57dc3 (Safwan)
Rooms, voting, role assignment, powers, endgame (first engine)	Before mid progress	2026-06-01 to 06-04	e597257, e51f3f1, 2709aef (Wasi)
Room flow merged	Before mid progress	2026-06-16	PR #4 (Safwan)
SRS session	2026-07-16	2026-07-16	Requirements workbook, /doc/srs.pdf
Design session	2026-07-22	2026-07-22	Design report (26 July), /doc/design.pdf
Themes, profiles, sound effects, engine reorganisation	After mid progress	2026-07-30	43dcd99, 7b5bc92, dc23e84 (Safwan)
Phase 0 audit; pure engine M1–M4; game service; reconnection; rate limits	Before Implementation session	2026-10-26	be25378, 66e3aa4, 0fce466, 615cc82

Milestone	Planned	Actual (from git)	Evidence
Voice chat, Royal Decree, music, admin dashboard, Docker, Render and Vercel	Before final presentation	2026-10-27	316941c, d5e2dd4, c534ff0, e3f90a9
Final presentation and live demo	2026-10-28	2026-10-28	Six-player live demo
Final report and repository	2026-11-01	Due	

SPL-II sessions: SRS on 16 July 2026, Design on 22 July 2026 (design report dated 26 July), and Implementation on 28 October 2026, the same day as the final presentation.

Where the work landed in time. Commits came in bursts: 18 in the week of 19 May, 5 in early June, 1 in mid-June, 5 in late July, then 189 in the week of 26 October. That last week holds 87% of all commits. The early bursts built a first working version. The final week rebuilt the engine after the Phase 0 audit found it was not safe or correct enough, and integrated the features the supervisor and advisor requested. The lesson is to schedule the audit and integration earlier, not in the final week (Section 11).

Task tracking evidence

- **Team workbook:** the single source of truth. It holds ownership, a task board of 31 tasks with owner, status, date landed and commit evidence, a decision log, and milestones.
- **Work rules:** pull before starting, branch from `dev`, never push to `main`, and log every session.
- **Module plan:** after the Phase 0 audit, the remaining work was split into 11 modules (M1–M11), each with its own branch (`feat/m1-game-engine` … `feat/m11-deployment-prod`).
- **`PROGRESS.md`:** a dated log of each session's branch, changes and next steps.

The task board from the team workbook (38 tasks, 13 per member, 3 team-wide):

Task board

Each member owns or co-owns 13 tasks, and three are team-wide. The open items are UI for features whose API is finished, a security fix, video chat, and features deferred to future work. "Landed" is the date the work first reached the repository.

#	Task	Owner	Status	Landed	Evidence
1	Monorepo scaffold (pnpm, Turborepo), docker-compose	Saif	Done	2026-05-19	a7babb1
2	CI pipeline: typecheck, test, build	Saif	Done	2026-05-19	a7babb1
3	Deploy hooks: API to Render, web to Vercel	Saif	Done	2026-05-19	6c68f1a
4	Login, sign-up and guest play (Better Auth)	Safwan	Done	2026-05-20	PR #1, 3f9267a
5	Admin app shell with sign-in	Safwan	Done	2026-05-20	3f9267a
6	Real-time chat	Safwan	Partial	2026-05-21	PR #3, 5f57dc3; history not shown on join

#	Task	Owner	Status	Landed	Evidence
7	First rooms: create and join by code	Safwan, Wasi	Done	2026-06-01	e597257, PR #4
8	First engine: voting, role assignment, powers, endgame	Wasi	Done	2026-06-04	e51f3f1, 2709aef; replaced by task 13
9	Themes, per-room chats, profiles, status indicators	Safwan	Done	2026-07-30	43dcd99
10	8-bit sound effects	Safwan	Done	2026-07-30	7b5bc92
11	Reorganise the first engine and room handler	Safwan	Done	2026-07-30	dc23e84
12	Phase 0 audit and module plan (M1–M11)	Wasi	Done	2026-10-26	Branches feat/m1 … feat/m11
13	Pure game engine M1–M4: reduce, roles 6–15, ballot, vision, abilities, endgame, scoring	Wasi	Done	2026-10-26	be25378, 9710bfe, 20a01c3, dae8c7e, 3606ee8
14	Leak tests: no role leaks through viewFor	Wasi	Done	2026-10-26	c1d9c1b
15	Game service: Redis snapshots, timers, restore	Wasi	Done	2026-10-26	66e3aa4
16	Data layer: Prisma schema, match recorder at game end	Safwan, Wasi	Done	—	packages/db, apps/api/src/persistence
17	Reconnection: 30 s grace, then auto-act	Saif	Done	2026-10-26	0fce466
18	Room routes on session identity; rate limiting	Wasi	Done	2026-10-26	7d66f68, 615cc82
19	Web room page on game:view / game:action	Saif	Done	2026-10-27	942daed
20	Users module: profiles, XP and stats	Saif	Partial	2026-10-27	ced1352; no avatar picker, history not shown
21	API test suite (19 files, 166 tests)	Saif, Wasi	Done	—	apps/api/src
22	Admin dashboard: telemetry, matches, users, rooms	Saif	Done	2026-10-27	65965e5, e3f90a9
23	Docker image and	Safwan,	Done	2026-10-	Docker branch, ec2a9b1

	render.yaml	Saif		27	

#	Task	Owner	Status	Landed	Evidence
24	Royal Decree custom rooms, pause and resume (supervisor request)	Wasi	Done	2026-10-27	d5e2dd4, 12a3ac9, PR #7
25	Voice chat, WebRTC peer-to-peer (advisor request)	Wasi	Done	2026-10-27	316941c
26	Background music (supervisor request)	Wasi	Partial	2026-10-27	c534ff0; not yet phase-adaptive
27	Vercel Git integration and deploy fixes	Wasi	Done	2026-10-27	PR #6, d363498
28	Production sign-in fixes (cross-site cookies)	Saif	Done	2026-10-28	834b2b4 to fa8fdf9
29	Mobile app	Safwan	Partial	—	Sign-in and global chat only
30	Host kick and ban buttons in web UI	Saif	Not started	—	API done: POST /rooms/:roomId/kick
31	Public room list and Quick Join in web UI	Saif	Not started	—	API done: GET /rooms, POST /rooms/quick-join
32	Chat history on join; profanity filter	Safwan	Not started	—	Follow-up to task 6
33	Google sign-in	Safwan	Not started	—	Future work
34	Video chat beside voice chat (WebRTC)	Saif	Not started	—	Future work
35	Re-enable CSRF and origin checks	Safwan	Not started	—	Security gap, report Section 8
36	Repository reorganisation (/code, /doc) and README	Team	In progress	2026-10-28	chore/repo-reorganisation
37	Final report	Team	In progress	—	Due 2026-11-01
38	Achievements, socials, feedback, email reports, push notifications	Team	Deferred	—	Future work

The decision log from the same workbook, newest first:

When	Decision	Why	Raised by
2026-10-28	Reorganise the repository into `/code` and `/doc`, with a README for visitors	Required by the SPL-II final submission notice	Course notice
2026-10-27	Deploy the web app through Vercel's Git integration instead of a CI job	The Hobby plan blocked deploys of commits by non-owners, and the CI job hung (PR #6)	Wasi
2026-10-26	Room routes take the player's identity from the session, never from the request	The audit found the old routes trusted a client-sent userId, so anyone could act as any player	Wasi
2026-10-26	Rebuild the game as a pure engine (`reduce`) with server timers, and delete the legacy socket-handler engine	The Phase 0 audit found the old engine wrote to Postgres on every action, had no server timers, leaked roles, ended the game after 3 rejections and dealt wrong role mixes at 14–15 players	Wasi, agreed by team
Before final presentation	Add peer-to-peer voice chat	In a social deduction game, players need to talk, not only type	Advisor
Before final presentation	Add background music and a custom room mode (Royal Decree)	Atmosphere, and host control over timers, roles and seats	Supervisor
Design session	Keep Redis for live game snapshots	Examiners asked why not keep game state only in server memory. The live state is in memory for speed; the Redis copy lets a game survive a server restart or redeploy and lets reconnecting players resync	Team, after examiner question

When	Decision	Why	Raised by
Design stage	Use a TCP Redis client (ioredis), with an in-memory fallback for local work	The Upstash REST API cannot do pub/sub or run the Socket.IO adapter	Team
SRS stage	The game design document is the source of truth for role rules; conflicting user stories were rewritten (Saint Frais, Jean Law, de Bussy, Siraj)	The spreadsheet and the design document disagreed	Team
SRS stage	Saint Frais' and Yar Lutuf's double vote applies to the ballot only, not to mission cards	Keeps missions readable and the double vote secret	Team
SRS stage	v1 scope = plan items 1–20 (a full playable match); everything else is post-mid	Examiners worried the scope was too large; secure the core first	Team, after examiner concern
SRS stage	Test backend and web first; defer the mobile app	Errors in the React Native build	Team

Scope and schedule changes

Change	Decision and why	How it was communicated
Scope concern from the panel	Fixed v1 to a complete playable match (items 1–20); extras became post-mid	Decision log; supervisor meeting
Mobile app deprioritised	React Native build errors; test backend and web first	Workbook note; group chat

Change	Decision and why	How it was communicated
Role stories rewritten	The game design document became the source of truth (Section 4)	Decision log
Engine rebuilt	The Phase 0 audit found security and rule problems in the first engine; rebuilt as a pure engine and deleted the old one	Phase 0 audit, module branches, group chat
Voice chat added	Advisor: a social deduction game needs voice	Advisor meeting; decision log
Background music and custom rooms added	Supervisor request	Supervisor meeting; decision log
Extras dropped from this release	Achievements, socials, feedback, email reports, push notifications and Google sign-in moved to future work	Task board status "Deferred" or "Not started"

Cost and effort estimates

Item	Service / tier	Cost
API hosting	Render, free web service (sleeps when idle)	0 BDT
Web and admin hosting	Vercel, Hobby	0 BDT
PostgreSQL	Neon, free tier	0 BDT
Redis	Upstash, free tier	0 BDT
Source control and CI	GitHub, free; GitHub Actions free minutes	0 BDT
Voice	WebRTC peer-to-peer with public STUN servers	0 BDT
Total infrastructure		**0 BDT per month**

If the game had real players, the first paid upgrades would be:

- an always-on Render instance, so the server does not sleep;
- a paid Neon or Upstash plan, for more storage and requests;
- a TURN server for players whose networks block peer-to-peer voice.

Effort. Each member worked 4–5 hours a week through the term (about 23 weeks, 19 May to 28 October), plus 6–8 hours before each SPL-II session (SRS, Design, and Implementation with the final presentation). That is about 110–140 hours per person and 330–420 person-hours for the team, including meetings, design and documentation as well as coding.

Feasibility

The plan was feasible within one semester and zero budget. Every part of the stack is free for our scale, the whole system uses one language (TypeScript), and the riskiest part (the rules engine) was isolated as a pure package that could be built and tested without the rest. The working six-player demo on 28 October 2026 confirms it.

10. Teamwork and Individual Contributions

The team agreed an equal split of the work and divided it along module boundaries, so each member owned complete parts of the system. Git history (commits by author) is the evidence for each member's work.

Member	ID	Responsibilities	Key evidence
Wasi Omar Syed	230042143	First rooms, voting, role assignment, powers and endgame (June); Phase 0 audit and module plan; the pure game engine (M1–M4: FSM, ballot, missions, vision, abilities, endgame, scoring) and its leak tests; server-side game service with Redis snapshots and timers; room routes on session identity; rate limiting; Royal Decree mode; voice chat; background music; production deploy fixes	109 commits on main; PRs #6–#9; be25378 (engine core), c1d9c1b (leak test), 66e3aa4 (game service), 316941c (voice)
Md. Saif-Al Sarker	230042106	Monorepo scaffold, CI workflow, docker-compose and deploy hooks; reconnection (30 s grace and automatic actions); wiring the web room page to `game:view` and `game:action`; users module; admin dashboard; final authentication fixes	82 commits on main; PR #2; a7babb1 (scaffold and CI), 0fce466 (reconnection), 942daed (web integration), e3f90a9 (admin)
M Safwan Hasan Khan	230042117	Better Auth login, sign-up and guests; real-time chat; first room flow; reorganisation of the first engine; DaisyUI themes, per-room chats, profiles, status indicators; 8-bit sound effects; Docker branch	10 commits on main; PRs #1, #3, #4; 3f9267a (auth), 5f57dc3 (chat), 43dcd99 (themes and profiles)
All three		Requirements, design, play-testing, documentation, presentation	Workbook, SRS, design document

Commit counts are not a measure of effort: Safwan's work came in a few large commits (about 22,700 lines added on main), while Wasi's and Saif's came in many small ones.

Coordinating interdependent modules

- **Contracts first.** The engine's `Action`, `PlayerView` and event types are exported from the engine package, and socket payloads are pinned down by Zod schemas. Saif wired the web room page to those types while the engine was being finished, and the web app later switched to the engine's own view types (401ba09).
- **A pure engine as the seam.** Because the engine does no input/output, it could be built and tested alone. The game service only calls `reduce()`, saves the snapshot and broadcasts views. Saif's reconnection tracker plugs into the same service: when the grace period ends, it asks the engine to act for the player.
- **Module branches.** Each module (M1–M11) had its own branch from `dev`, so members worked in parallel without blocking each other.
- **Shared record.** The team workbook (ownership, task board, decision log) and `PROGRESS.md` told everyone what had changed after each session.

Disagreements and blockers

Issue	How it was resolved
Whether to add voice chat at all: extra complexity late in the term vs. a better game	The advisor pointed out that social deduction is played by talking. The team agreed and chose peer-to-peer WebRTC, which keeps it off our server
The requirements spreadsheet and the game design document disagreed on several roles	The team agreed the game design document is the source of truth and rewrote the conflicting stories
Redis questioned by the evaluation panel	Discussed in the team; kept Redis for restart survival and reconnection, with an in-memory fallback (Section 5)
The first engine was found unsafe in the Phase 0 audit	Agreed to rebuild it as a pure engine instead of patching it, and split the work into modules M1–M11
The mobile build had errors	Deferred mobile; tested backend and web first

Issue	How it was resolved
Vercel's Hobby plan blocked deploys of commits by non-owners	Switched the web deploy to Vercel's Git integration (PR #6)

Decisions were made in meetings with the supervisor and in the team group chat, then recorded in the workbook's decision log.

Fair participation and inclusion

- **Every member owns complete subsystems.** Wasi owns the game engine and game service, Saif owns the platform (scaffold, CI, reconnection, admin) and Safwan owns accounts and the social layer (auth, chat, themes, profiles, sound). Each subsystem can be traced to its owner in git.
- **Similar effort, different commit styles.** Each member gave about 4–5 hours a week through the term, plus 6–8 hours before each presentation. Commit counts differ (109, 82, 10) because of commit size, not effort: lines added on main are of the same order for all three.
- **Shared documents.** The SRS, design document, presentation and this report were written together, and every member presented their own parts.
- **Uneven timing.** Most commits (189 of 218) landed in the final week. This was a team-wide pattern, not one member's, and we discuss it as a lesson in Section 11.

11. Results, Limitations, and Future Work

We checked the finished system against the 19 functional requirements in our SRS by reading the code, not the plans. 13 are complete and 6 are partly done. None were dropped.

Status of each requirement

Requirement	Status	Notes
FR_1 Lobby creation and customisation	Complete	6-character codes, player limit, Royal Decree role toggles and timers
FR_2 Matchmaking and joining	Partial	Join by code works; the public room list and Quick Join exist in the API but not in the web UI

Requirement	Status	Notes
FR_3 Public communication	Partial	Room and global chat work; chat history is not shown on join, chat is not phase-aware and there is no profanity filter
FR_4 Secret role reveal	Complete	Each player sees only their own role through `viewFor`
FR_5 Faction vision	Complete	Siraj's vision, Red mutual recognition and Mir Madan's Round 1 rule, covered by leak tests
FR_6 Discussion timer	Complete	Server-side timers, configurable per round in Royal Decree
FR_7 Ballot voting and fail track	Complete	Ties reject; five rejections hand the round to Red
FR_8 Abilities	Complete	de Bussy, Jean Law and Saint Frais powers enforced by the engine
FR_9 Mission cards	Complete	Secret Success or Sabotage, validated by role
FR_10 Host kick and ban	Partial	Implemented in the API; no kick or ban button in the web UI
FR_11 Registration, login, sessions, guests	Partial	Email and password, sessions and guests work; no Google sign-in; username can only be set after sign-up
FR_12 Profiles, stats, avatars	Partial	Level, XP, matches and win rate shown; no avatar picker and no match-history list in the UI
FR_13 Leader rotation and nominations	Complete	Rotation and nomination enforced by the engine
FR_14 Results and victory conditions	Complete	Round wins, endgame guesses and Jagat Seth's solo win
FR_15 Post-game report	Complete	All roles revealed with win or loss per player

Requirement	Status	Notes
FR_16 Audio-visual	Partial	Three background tracks and sound effects; music does not change with the game phase
FR_17 Reconnection	Complete	30-second grace period, then automatic actions
FR_18 Server-side validation	Complete	Every action checked by the engine and Zod schemas
FR_19 Host migration	Complete	The host role passes to the next player when the host leaves

Features planned after the mid evaluation but not built: achievements, social features (friends), a feedback form, email reports and push notifications. The mobile app has sign-in and global chat only; the game itself is played on the web.

Known limitations

Limitations

- **Security gaps.** CSRF and origin checks are turned off to make cross-origin cookies work, and one statistics endpoint needs no sign-in (Section 8).
- **Missing UI for working features.** Kick, ban, Quick Join and the public room list work in the API but have no buttons in the web app.
- **Chat history.** Players who join a room do not see earlier messages.
- **No Google sign-in**; accounts use email and password.
- **Testing.** 419 automated tests pass, but there is no coverage measurement, no web front-end tests, no end-to-end or load tests, and API tests use a mocked database.
- **Process.** CI ran on every push but was not required before merging, most commits went straight to `main`, and most of the work was done in the final week.

Future work

Future work

1. Turn CSRF and origin checks back on with an explicit list of trusted origins.
2. Protect `main`: require a pull request and a passing CI run before merging.

3. Add the missing UI: kick and ban buttons, the public room list and Quick Join, chat history on join, an avatar picker and match history.
4. Add Google sign-in, account deletion and data export.
5. Add video chat beside the existing voice chat, peer-to-peer over WebRTC like voice, so players can read each other's faces during discussion.
6. Measure coverage, and add web component tests and an end-to-end test of a full game.
7. Make the music follow the game phase.
8. Finish the mobile app so the game can be played on phones.
9. Add an MIT `LICENSE` file so others can reuse the code with credit.

12. Glossary and References

Glossary

Term	Meaning
API	Application Programming Interface; here, the game server other apps talk to
Ballot	The vote in which every player approves or rejects the Leader's proposed team
CI/CD	Continuous Integration / Continuous Deployment: automatically testing and deploying code on every change
DTO / data class	A plain object shape sent between components, such as `game:action` or `PlayerView`
EIC	The British East India Company; the Red side
Endgame	The final guessing phase after a side wins 3 rounds
Fail track	The counter of rejected teams in a round; at 5, the round goes to Red
FSM	Finite State Machine: a fixed set of states and allowed moves between them
Minister	A player chosen for the current mission team

Term	Meaning
Monorepo	One repository holding several apps and shared packages
ORM	Object-Relational Mapper; Prisma turns database tables into typed objects
Pure function	A function whose output depends only on its inputs and which changes nothing outside itself
Redis	A fast in-memory data store, used for live game snapshots
REST	Ordinary request-response web calls over HTTP
Seeded RNG	A random-number generator that gives the same sequence for the same seed, so games can be replayed
Socket.IO / WebSocket	A connection kept open so the server can push updates instantly
viewFor	Our function that builds what one player is allowed to see
WebRTC	Browser technology for direct peer-to-peer audio between players
Zod	A library that checks incoming data matches an expected shape

References

1. Battle of Plassey. Wikipedia. https://en.wikipedia.org/wiki/Battle_of_Plassey (accessed 28 Oct 2026)
2. Mafia (party game). Wikipedia. https://en.wikipedia.org/wiki/Mafia_(party_game) (accessed 28 Oct 2026)
3. The Resistance (game). Wikipedia. https://en.wikipedia.org/wiki/The_Resistance_(game) (accessed 28 Oct 2026)
4. Secret Hitler. Wikipedia. https://en.wikipedia.org/wiki/Secret_Hitler (accessed 28 Oct 2026)

5. Town of Salem. Wikipedia. https://en.wikipedia.org/wiki/Town_of_Salem (accessed 28 Oct 2026)
6. Among Us. Wikipedia. https://en.wikipedia.org/wiki/Among_Us (accessed 28 Oct 2026)
7. Group 3. *1757: Software Requirements Specification (requirements workbook, 16 July 2026)*. `/doc/srs.pdf`
8. Group 3. *1757: Software Design Report (26 July 2026)*. `/doc/design.pdf`
9. Group 3. *1757: Final Presentation*. `/doc/final-presentation.pdf`
10. Official documentation: Socket.IO, Fastify, Prisma, Better Auth, Next.js, Expo, Zod, Vitest, Turborepo, WebRTC on MDN.